SALES & CUSTOMER ANALYTICS DOCUMENT

# NAILVERSE

*Not just NAILS.. IT's the WHOLE MOOD*

*Online Nail Art & Nail Beauty Online Platform*

***WEEK - 9***

---

## 9. Sales and Customer Analytics System

### Scenario

A month of bookings, payments and reviews is now stored in the Nailverse database. The owners want to turn this data into answers: who are the customers that spend the most, which services sell best, which categories bring in the money, and which services need a little promotion.

This is done with SQL aggregate functions together with GROUP BY, HAVING and joins. The results are collected into four business analytics reports at the end of the week.

### 1. Apply SQL Aggregate Functions

An aggregate function works on many rows at once and gives back one summarised value.

| Function | Purpose | Nailverse Example |
|---|---|---|
| COUNT() | Counts the number of records | Total customers / appointments |
| SUM() | Adds up the values | Total sales amount |
| AVG() | Finds the average value | Average appointment value |
| MIN() | Finds the smallest value | Lowest service price |
| MAX() | Finds the largest value | Highest service price |

### 2. Generate Total Sales Reports

Tasks: Write SQL queries to calculate:

1. The total number of appointments booked.

```
SQL> SELECT COUNT(*) AS Number_Appointments_Book FROM Appointment;

NUMBER_APPOINTMENTS_BOOK
------------------------
                      10
```

2. The total revenue generated from sales.

```
SQL> SELECT SUM(Total_Amount) AS Total_Revenue FROM Appointment;

TOTAL_REVENUE
-------------
        13599
```

3. The average appointment value.

```
SQL> SELECT AVG(Total_Amount) AS Average_Appointment
  2  FROM Appointment;

AVERAGE_APPOINTMENT
-------------------
             1359.9
```

GROUP BY:

```
SQL> SELECT AVG(Total_Amount) AS AVERAGE FROM Appointment GROUP BY Appointm
ent_ID;

   AVERAGE
----------
       999
      1200
      1500
       800
      2000
       750
      1800
       950
      1100
      2500
```

4. The highest and lowest appointment amount.

```
SQL> SELECT MAX(Total_Amount) AS Highest_Appointment,
  2  MIN(Total_Amount) AS Lowest_Appointment
  3  FROM Appointment;

HIGHEST_APPOINTMENT LOWEST_APPOINTMENT
------------------- ------------------
               2500                750
```

5. The total sales for a specific period (1 to 7 September 2026).

```
Connected.
SQL> SELECT SUM(Total_Amount) AS Total_Revenue FROM Appointment
  2  WHERE Appointment_Date BETWEEN DATE '2026-06-28' AND DATE '
2026-10-26';

TOTAL_REVENUE
-------------
         1757
```

## 3. Customer Purchase Analysis

1. Total number of appointments booked by each customer.

```
SQL> SELECT c.Customer_ID,
  2         c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  3         COUNT(a.Appointment_ID) AS Total_Appointments
  4  FROM Customer c
  5  LEFT JOIN Appointment a
  6      ON c.Customer_ID = a.Customer_ID
  7  GROUP BY c.Customer_ID, c.First_Name, c.Last_Name
  8  ORDER BY Total_Appointments DESC, c.Customer_ID;

CUSTOMER_ID          CUSTOMER_NAME
-------------------- ----------------------------------------
TOTAL_APPOINTMENTS
------------------
CU-1001              MEERA IYER
                  1

CU-1002              KAVYA RAO
                  1

CU-1003              ANANYA  SURESH
                  1


CUSTOMER_ID          CUSTOMER_NAME
-------------------- ----------------------------------------
TOTAL_APPOINTMENTS
------------------
CU-1004              PRIYA NAIR
                  1

CU-1005              NISHA KUMARI
                  1

CU-1006              VARSHA RAJAN
                  1


CUSTOMER_ID          CUSTOMER_NAME
-------------------- ----------------------------------------
TOTAL_APPOINTMENTS
------------------
CU-1007              MOSHAMI JUN
                  1

CU-1008              HARI KUMAR
                  1

CU-1009              SRI VALLI
                  1


CUSTOMER_ID          CUSTOMER_NAME
-------------------- ----------------------------------------
TOTAL_APPOINTMENTS
------------------
CU-1010              NILLA VEERA
                  1


10 rows selected.
```

2. Total amount spent by each customer.

```
SQL> SELECT c.Customer_ID,
  2         c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  3         NVL(SUM(a.Total_Amount), 0) AS Total_Amount_Spent
  4  FROM Customer c
  5  LEFT JOIN Appointment a
  6      ON c.Customer_ID = a.Customer_ID
  7  GROUP BY c.Customer_ID, c.First_Name, c.Last_Name
  8  ORDER BY Total_Amount_Spent DESC, c.Customer_ID;

CUSTOMER_ID          CUSTOMER_NAME
-------------------- ----------------------------------------
TOTAL_AMOUNT_SPENT
------------------
CU-1006              VARSHA RAJAN
                1299

CU-1004              PRIYA NAIR
                 999

CU-1010              NILLA VEERA
                 999


CUSTOMER_ID          CUSTOMER_NAME
-------------------- ----------------------------------------
TOTAL_AMOUNT_SPENT
------------------
CU-1005              NISHA KUMARI
                 799

CU-1002              KAVYA RAO
                 650

CU-1008              HARI KUMAR
                 650


CUSTOMER_ID          CUSTOMER_NAME
-------------------- ----------------------------------------
TOTAL_AMOUNT_SPENT
------------------
CU-1001              MEERA IYER
                 499

CU-1007              MOSHAMI JUN
                 499

CU-1003              ANANYA  SURESH
                 459


CUSTOMER_ID          CUSTOMER_NAME
-------------------- ----------------------------------------
TOTAL_AMOUNT_SPENT
------------------
CU-1009              SRI VALLI
                 459


10 rows selected.
```

The NVL() function replaces NULL with a chosen value. For a customer who has not booked anything, the amount spent is shown as zero instead of an empty cell.

3. Average spending per customer.

```
SQL> SELECT c.Customer_ID,
  2         c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  3         ROUND(NVL(AVG(a.Total_Amount), 0), 2) AS Average_Spending
  4  FROM Customer c
  5  LEFT JOIN Appointment a
  6      ON c.Customer_ID = a.Customer_ID
  7  GROUP BY c.Customer_ID, c.First_Name, c.Last_Name
  8  ORDER BY Average_Spending DESC, c.Customer_ID;

CUSTOMER_ID          CUSTOMER_NAME                              AVERAGE_SPENDING
-------------------- ---------------------------------------- ----------------
CU-1006              VARSHA RAJAN                                         1299
CU-1004              PRIYA NAIR                                            999
CU-1010              NILLA VEERA                                           999
CU-1005              NISHA KUMARI                                          799
CU-1002              KAVYA RAO                                             650
CU-1008              HARI KUMAR                                            650
CU-1001              MEERA IYER                                            499
CU-1007              MOSHAMI JUN                                           499
CU-1003              ANANYA  SURESH                                        459
CU-1009              SRI VALLI                                             459

10 rows selected.
```

4. Customers with the maximum purchase amount.

```
SQL> SELECT c.Customer_ID,
  2         c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  3         SUM(a.Total_Amount) AS Total_Purchase_Amount
  4  FROM Customer c
  5  JOIN Appointment a
  6      ON c.Customer_ID = a.Customer_ID
  7  GROUP BY c.Customer_ID, c.First_Name, c.Last_Name
  8  HAVING SUM(a.Total_Amount) = (
  9      SELECT MAX(Total_Spending)
 10      FROM (
 11          SELECT SUM(Total_Amount) AS Total_Spending
 12          FROM Appointment
 13          GROUP BY Customer_ID
 14      )
 15  );

CUSTOMER_ID          CUSTOMER_NAME
-------------------- ----------------------------------------
TOTAL_PURCHASE_AMOUNT
---------------------
CU-1006              VARSHA RAJAN
                    1299
```

5. Customers with fewer bookings (2 appointments or less).+__

```
SQL> SELECT c.Customer_ID,
  2         c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  3         COUNT(a.Appointment_ID) AS Number_Of_Appointments
  4  FROM Customer c
  5  LEFT JOIN Appointment a
  6       ON c.Customer_ID = a.Customer_ID
  7  GROUP BY c.Customer_ID, c.First_Name, c.Last_Name
  8  HAVING COUNT(a.Appointment_ID) <= 2
  9  ORDER BY Number_Of_Appointments;

CUSTOMER_ID          CUSTOMER_NAME
-------------------- ----------------------------------------
NUMBER_OF_APPOINTMENTS
----------------------
CU-1004              PRIYA NAIR
                     1

CU-1006              VARSHA RAJAN
                     1

CU-1007              MOSHAMI JUN
                     1


CUSTOMER_ID          CUSTOMER_NAME
-------------------- ----------------------------------------
NUMBER_OF_APPOINTMENTS
----------------------
CU-1009              SRI VALLI
                     1

CU-1001              MEERA IYER
                     1

CU-1010              NILLA VEERA
                     1


CUSTOMER_ID          CUSTOMER_NAME
-------------------- ----------------------------------------
NUMBER_OF_APPOINTMENTS
----------------------
CU-1005              NISHA KUMARI
                     1

CU-1003              ANANYA  SURESH
                     1

CU-1008              HARI KUMAR
                     1


CUSTOMER_ID          CUSTOMER_NAME
-------------------- ----------------------------------------
NUMBER_OF_APPOINTMENTS
----------------------
CU-1002              KAVYA RAO
                     1


10 rows selected.
```

## 4. Find Top Customers Based on Purchase Amount

1. Top 5 customers based on total spending.

```
SQL> SELECT *
  2  FROM (
  3      SELECT c.Customer_ID,
  4             c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  5             SUM(a.Total_Amount) AS Total_Spending
  6      FROM Customer c
  7      JOIN Appointment a
  8          ON c.Customer_ID = a.Customer_ID
  9      GROUP BY c.Customer_ID, c.First_Name, c.Last_Name
 10      ORDER BY Total_Spending DESC
 11  )
 12  WHERE ROWNUM <= 5;

CUSTOMER_ID          CUSTOMER_NAME                            TOTAL_SPENDING
-------------------- ---------------------------------------- --------------
CU-1006              VARSHA RAJAN                                       1299
CU-1004              PRIYA NAIR                                          999
CU-1010              NILLA VEERA                                         999
CU-1005              NISHA KUMARI                                        799
CU-1002              KAVYA RAO                                           650
```

2. Customers with the maximum number of appointments.

```
SQL> SELECT c.Customer_ID,
  2         c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  3         COUNT(a.Appointment_ID) AS Number_Of_Appointments
  4  FROM Customer c
  5  JOIN Appointment a
  6      ON c.Customer_ID = a.Customer_ID
  7  GROUP BY c.Customer_ID, c.First_Name, c.Last_Name
  8  HAVING COUNT(a.Appointment_ID) = (
  9      SELECT MAX(Appointment_Count)
 10      FROM (
 11          SELECT COUNT(*) AS Appointment_Count
 12          FROM Appointment
 13          GROUP BY Customer_ID
 14      )
 15  )
 16  ORDER BY Customer_Name;

CUSTOMER_ID          CUSTOMER_NAME
-------------------- ----------------------------------------
NUMBER_OF_APPOINTMENTS
----------------------
CU-1003              ANANYA  SURESH
                     1

CU-1008              HARI KUMAR
                     1

CU-1002              KAVYA RAO
                     1


CUSTOMER_ID          CUSTOMER_NAME
-------------------- ----------------------------------------
NUMBER_OF_APPOINTMENTS
----------------------
CU-1001              MEERA IYER
                     1

CU-1007              MOSHAMI JUN
                     1

CU-1010              NILLA VEERA
                     1


CUSTOMER_ID          CUSTOMER_NAME
-------------------- ----------------------------------------
NUMBER_OF_APPOINTMENTS
----------------------
CU-1005              NISHA KUMARI
                     1

CU-1004              PRIYA NAIR
                     1

CU-1009              SRI VALLI
                     1


CUSTOMER_ID          CUSTOMER_NAME
-------------------- ----------------------------------------
NUMBER_OF_APPOINTMENTS
----------------------
CU-1006              VARSHA RAJAN
                     1


10 rows selected.
```

3. Frequent customers (3 or more appointments).

```
SQL> SELECT c.Customer_ID,
  2         c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  3         COUNT(a.Appointment_ID) AS Number_Of_Appointments
  4  FROM Customer c
  5  JOIN Appointment a
  6      ON c.Customer_ID = a.Customer_ID
  7  GROUP BY c.Customer_ID, c.First_Name, c.Last_Name
  8  HAVING COUNT(a.Appointment_ID) >= 3
  9  ORDER BY Number_Of_Appointments DESC, Customer_Name;

no rows selected
```

4. High-value customers (total spending above ₹4000).

```
SQL> SELECT c.Customer_ID,
  2         c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  3         COUNT(a.Appointment_ID) AS Number_Of_Appointments,
  4         SUM(a.Total_Amount) AS Total_Spending
  5  FROM Customer c
  6  JOIN Appointment a
  7      ON c.Customer_ID = a.Customer_ID
  8  GROUP BY c.Customer_ID, c.First_Name, c.Last_Name
  9  HAVING SUM(a.Total_Amount) > 700
 10  ORDER BY Total_Spending DESC;

CUSTOMER_ID          CUSTOMER_NAME
-------------------- ----------------------------------------
NUMBER_OF_APPOINTMENTS TOTAL_SPENDING
---------------------- --------------
CU-1006              VARSHA RAJAN
                     1           1299

CU-1010              NILLA VEERA
                     1            999

CU-1004              PRIYA NAIR
                     1            999


CUSTOMER_ID          CUSTOMER_NAME
-------------------- ----------------------------------------
NUMBER_OF_APPOINTMENTS TOTAL_SPENDING
---------------------- --------------
CU-1005              NISHA KUMARI
                     1            799
```

## 5. Identify Best-Selling Services

1. Services with the maximum sales quantity.

```
SQL> SELECT s.Service_ID,
  2         s.Service_Name,
  3         SUM(ad.Quantity) AS Total_Quantity_Sold
  4  FROM Service s
  5  JOIN Appointment_Details ad
  6      ON s.Service_ID = ad.Service_ID
  7  GROUP BY s.Service_ID, s.Service_Name
  8  ORDER BY Total_Quantity_Sold DESC, s.Service_Name;

SERVICE SERVICE_NAME                                       TOTAL_QUANTITY_SOLD
------- -------------------------------------------------- -------------------
SRV004  CHROME NAIL ART                                                      2
SRV001  CLASSIC MAINCURE                                                     2
SRV002  CLASSIC PEDICURE                                                     2
SRV003  NAIL EXTENSIONS                                                      2
SRV006  Chrome Nail Art                                                      1
SRV005  SPA PEDICURE                                                         1

6 rows selected.
```

2. Services generating the highest revenue.

```
SQL> SELECT s.Service_ID,
  2         s.Service_Name,
  3         SUM(ad.Quantity) AS Quantity_Sold,
  4         SUM(ad.Quantity * ad.Price) AS Total_Revenue
  5  FROM Service s
  6  JOIN Appointment_Details ad
  7      ON s.Service_ID = ad.Service_ID
  8  GROUP BY s.Service_ID, s.Service_Name
  9  ORDER BY Total_Revenue DESC, s.Service_Name;

SERVICE SERVICE_NAME                                       QUANTITY_SOLD
------- -------------------------------------------------- -------------
TOTAL_REVENUE
-------------
SRV004  CHROME NAIL ART                                                2
         1998

SRV002  CLASSIC PEDICURE                                               2
         1300

SRV006  Chrome Nail Art                                                1
         1299


SERVICE SERVICE_NAME                                       QUANTITY_SOLD
------- -------------------------------------------------- -------------
TOTAL_REVENUE
-------------
SRV001  CLASSIC MAINCURE                                               2
          998

SRV003  NAIL EXTENSIONS                                                2
          918

SRV005  SPA PEDICURE                                                   1
          799


6 rows selected.
```

3. Least-selling services.

```
SQL> SELECT s.Service_ID,
  2         s.Service_Name,
  3         NVL(SUM(ad.Quantity), 0) AS Quantity_Sold
  4  FROM Service s
  5  LEFT JOIN Appointment_Details ad
  6      ON s.Service_ID = ad.Service_ID
  7  GROUP BY s.Service_ID, s.Service_Name
  8  ORDER BY Quantity_Sold ASC, s.Service_Name;

SERVICE SERVICE_NAME                                       QUANTITY_SOLD
------- -------------------------------------------------- -------------
SRV006  Chrome Nail Art                                                1
SRV005  SPA PEDICURE                                                   1
SRV004  CHROME NAIL ART                                                2
SRV001  CLASSIC MAINCURE                                               2
SRV002  CLASSIC PEDICURE                                               2
SRV003  NAIL EXTENSIONS                                                2

6 rows selected.
```

4. Services requiring promotion (booked once or never, but with free slots).

```
SQL> SELECT s.Service_ID,
  2         s.Service_Name,
  3         s.Price,
  4         NVL(SUM(ad.Quantity), 0) AS Quantity_Sold
  5  FROM Service s
  6  LEFT JOIN Appointment_Details ad
  7      ON s.Service_ID = ad.Service_ID
  8  GROUP BY s.Service_ID, s.Service_Name, s.Price
  9  HAVING NVL(SUM(ad.Quantity), 0) <= 1
 10  ORDER BY Quantity_Sold, s.Service_Name;

SERVICE SERVICE_NAME                                         PRICE
------- -------------------------------------------------- ----------
QUANTITY_SOLD
-------------
SRV006  Chrome Nail Art                                        1299
          1

SRV005  SPA PEDICURE                                            799
          1
```

## 6. Perform Category-wise Sales Analysis

1. Total sales generated by each category.

```
SQL> SELECT c.Category_Name,
  2         SUM(ad.Quantity * ad.Price) AS Total_Sales
  3  FROM Category c
  4  JOIN Service s
  5      ON c.Category_ID = s.Category_ID
  6  JOIN Appointment_Details ad
  7      ON s.Service_ID = ad.Service_ID
  8  GROUP BY c.Category_ID, c.Category_Name
  9  ORDER BY Total_Sales DESC;

CATEGORY_NAME                 TOTAL_SALES
----------------------------- -----------
NAIL EXTENSIONS                      2217
NAIL ART                             1998
PEDICURE                             1300
MANICURE                              998
SPA CARE                              799
```

2. Number of services sold in each category.

```
SQL> SELECT c.Category_Name,
  2         SUM(ad.Quantity) AS Services_Sold
  3  FROM Category c
  4  JOIN Service s
  5      ON c.Category_ID = s.Category_ID
  6  JOIN Appointment_Details ad
  7      ON s.Service_ID = ad.Service_ID
  8  GROUP BY c.Category_ID, c.Category_Name
  9  ORDER BY Services_Sold DESC, c.Category_Name;

CATEGORY_NAME                 SERVICES_SOLD
----------------------------- -------------
NAIL EXTENSIONS                           3
MANICURE                                  2
NAIL ART                                  2
PEDICURE                                  2
SPA CARE                                  1
```

3. Highest revenue-generating category.

```
SQL> SELECT c.Category_Name,
  2         SUM(ad.Quantity * ad.Price) AS Total_Revenue
  3  FROM Category c
  4  JOIN Service s ON c.Category_ID = s.Category_ID
  5  JOIN Appointment_Details ad ON s.Service_ID = ad.Service_ID
  6  GROUP BY c.Category_ID, c.Category_Name
  7  HAVING SUM(ad.Quantity * ad.Price) = (
  8      SELECT MAX(Category_Revenue)
  9      FROM (
 10          SELECT SUM(ad2.Quantity * ad2.Price) AS Category_Revenue
 11          FROM Service s2
 12          JOIN Appointment_Details ad2
 13              ON s2.Service_ID = ad2.Service_ID
 14          GROUP BY s2.Category_ID
 15      )
 16  );

CATEGORY_NAME                  TOTAL_REVENUE
------------------------------ -------------
NAIL EXTENSIONS                         2217
```

4. Average sales per category.

```
SQL> SELECT c.Category_Name,
  2         ROUND(AVG(ad.Quantity * ad.Price), 2) AS Average_Sales
  3  FROM Category c
  4  JOIN Service s ON c.Category_ID = s.Category_ID
  5  JOIN Appointment_Details ad ON s.Service_ID = ad.Service_ID
  6  GROUP BY c.Category_ID, c.Category_Name
  7  ORDER BY Average_Sales DESC;

CATEGORY_NAME                 AVERAGE_SALES
----------------------------- -------------
NAIL ART                                999
SPA CARE                                799
NAIL EXTENSIONS                         739
PEDICURE                                650
MANICURE                                499
```

## 7. Generate Business Analytics Reports

### Report 1: Sales Performance Report

- Total sales
- Number of appointments
- Average appointment value
- Highest appointment value

```
SQL> SELECT SUM(Total_Amount) AS Total_Sales,
  2         COUNT(Appointment_ID) AS Number_Of_Appointments,
  3         ROUND(AVG(Total_Amount), 2) AS Average_Appointment_Value,
  4         MAX(Total_Amount) AS Highest_Appointment_Value
  5  FROM Appointment;

TOTAL_SALES NUMBER_OF_APPOINTMENTS AVERAGE_APPOINTMENT_VALUE
----------- ---------------------- -------------------------
HIGHEST_APPOINTMENT_VALUE
-------------------------
       7312                     10                    731.2
                   1299
```

### Report 2: Customer Analytics Report

- Customer name
- Number of appointments
- Total spending
- Booking frequency

```
SQL> SELECT c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  2         COUNT(a.Appointment_ID) AS Number_Of_Appointments,
  3         NVL(SUM(a.Total_Amount), 0) AS Total_Spending,
  4         CASE
  5           WHEN COUNT(a.Appointment_ID) >= 3 THEN 'Frequent'
  6           WHEN COUNT(a.Appointment_ID) >= 1 THEN 'Occasional'
  7           ELSE 'No Booking'
  8         END AS Booking_Frequency
  9  FROM Customer c
 10  LEFT JOIN Appointment a
 11    ON c.Customer_ID = a.Customer_ID
 12  GROUP BY c.Customer_ID, c.First_Name, c.Last_Name
 13  ORDER BY Total_Spending DESC;

CUSTOMER_NAME                          NUMBER_OF_APPOINTMENTS TOTAL_SPENDING
-------------------------------------- ---------------------- --------------
BOOKING_FR
----------
VARSHA RAJAN                                               1           1299
Occasional

PRIYA NAIR                                                 1            999
Occasional

NILLA VEERA                                                1            999
Occasional


CUSTOMER_NAME                          NUMBER_OF_APPOINTMENTS TOTAL_SPENDING
-------------------------------------- ---------------------- --------------
BOOKING_FR
----------
NISHA KUMARI                                               1            799
Occasional

KAVYA RAO                                                  1            650
Occasional

HARI KUMAR                                                 1            650
Occasional
```

```
CUSTOMER_NAME                          NUMBER_OF_APPOINTMENTS TOTAL_SPENDING
-------------------------------------- ---------------------- --------------
BOOKING_FR
----------
MOSHAMI JUN                                                1            499
Occasional

MEERA IYER                                                 1            499
Occasional

ANANYA  SURESH                                             1            459
Occasional


CUSTOMER_NAME                          NUMBER_OF_APPOINTMENTS TOTAL_SPENDING
-------------------------------------- ---------------------- --------------
BOOKING_FR
----------
SRI VALLI                                                  1            459
Occasional


10 rows selected.
```

### Report 3: Service Performance Report

- Service name
- Quantity sold
- Revenue generated

```
SQL> SELECT s.Service_Name,
  2         SUM(ad.Quantity) AS Quantity_Sold,
  3         SUM(ad.Quantity * ad.Price) AS Revenue_Generated
  4  FROM Service s
  5  JOIN Appointment_Details ad
  6    ON s.Service_ID = ad.Service_ID
  7  GROUP BY s.Service_ID, s.Service_Name
  8  ORDER BY Revenue_Generated DESC;

SERVICE_NAME                                       QUANTITY_SOLD
-------------------------------------------------- -------------
REVENUE_GENERATED
-----------------
CHROME NAIL ART                                                2
             1998

CLASSIC PEDICURE                                               2
             1300

Chrome Nail Art                                                1
             1299


SERVICE_NAME                                       QUANTITY_SOLD
-------------------------------------------------- -------------
REVENUE_GENERATED
-----------------
CLASSIC MAINCURE                                               2
              998

NAIL EXTENSIONS                                                2
              918

SPA PEDICURE                                                   1
              799


6 rows selected.
```

### Report 4: Category Analysis Report

- Category name
- Total services sold
- Total revenue

```
SQL> SELECT c.Category_Name,
  2         SUM(ad.Quantity) AS Total_Services_Sold,
  3         SUM(ad.Quantity * ad.Price) AS Total_Revenue
  4  FROM Category c
  5  JOIN Service s ON c.Category_ID = s.Category_ID
  6  JOIN Appointment_Details ad ON s.Service_ID = ad.Service_ID
  7  GROUP BY c.Category_ID, c.Category_Name
  8  ORDER BY Total_Revenue DESC;

CATEGORY_NAME                 TOTAL_SERVICES_SOLD TOTAL_REVENUE
----------------------------- ------------------- -------------
NAIL EXTENSIONS                                 3          2217
NAIL ART                                        2          1998
PEDICURE                                        2          1300
MANICURE                                        2           998
SPA CARE                                        1           799
```

### Conclusion

Using aggregate functions, GROUP BY, HAVING, joins and NVL(), the Nailverse data was turned into clear business information. The reports show the total and average sales, the customers who spend the most and book most often, the best-selling and least-booked services, and the categories that earn the most. This completes the Nailverse database project from requirement analysis to sales analytics.